

Q.1) Take multiple files as Command Line Arguments and print their file type and inode number

[10 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/stat.h>

void print_file_info(const char *filename) {
    struct stat st;

    if (stat(filename, &st) == -1) {
        perror(filename);
        return;
    }

    printf("File: %s\n", filename);
    printf("Inode number: %ld\n", st.st_ino);

    printf("File type: ");
    if (S_ISREG(st.st_mode)) printf("Regular file\n");
    else if (S_ISDIR(st.st_mode)) printf("Directory\n");
    else if (S_ISCHR(st.st_mode)) printf("Character device\n");
    else if (S_ISBLK(st.st_mode)) printf("Block device\n");
    else if (S_ISFIFO(st.st_mode)) printf("FIFO/pipe\n");
    else if (S_ISLNK(st.st_mode)) printf("Symbolic link\n");
    else if (S_ISSOCK(st.st_mode)) printf("Socket\n");
    else printf("Unknown\n");

    printf("\n");
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file1> <file2> ...\n", argv[0]);
```

```

        return 1;
    }

    for (int i = 1; i < argc; i++) {
        print_file_info(argv[i]);
    }

    return 0;
}

```

Q.2) Implement the following unix/linux command (use fork, pipe and exec system call)

ls -l | wc -l [20 Marks]

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

int main() {
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

```

```

    pid1 = fork();
    if (pid1 < 0) {
        perror("fork");
        exit(1);
    }

```

```

    if (pid1 == 0) {
        // First child: ls -l

```

```

close(fd[0]);          // Close read end
dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe
close(fd[1]);

execlp("ls", "ls", "-l", NULL);
perror("execlp ls");
exit(1);
}

pid2 = fork();
if (pid2 < 0) {
    perror("fork");
    exit(1);
}

if (pid2 == 0) {
    // Second child: wc -l
    close(fd[1]);          // Close write end
    dup2(fd[0], STDIN_FILENO); // Redirect stdin from pipe
    close(fd[0]);

    execlp("wc", "wc", "-l", NULL);
    perror("execlp wc");
    exit(1);
}

// Parent closes pipe ends
close(fd[0]);
close(fd[1]);

// Wait for children to finish
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

```

```
printf("Command executed successfully.\n");  
return 0;  
}
```